



Computational performance of the E3SM spectral element dynamical core

Mark Taylor

Sandia National Labs, mataylo@sandia.gov

E3SM Performance Group and CMDV-SM project:

L. Bertagna, M. Deakin, O. Guba, D. Sunderland, A. M. Bradley, I. K. Tezaur, A. G. Salinger, N. Keen, A. Mametjanov

CEED Project meeting, Boulder, Aug 9 2018

SAND2018-5112 PE

"E3SM will never run any faster than it does on NERSC Edison (~2012 Ivybridge)"

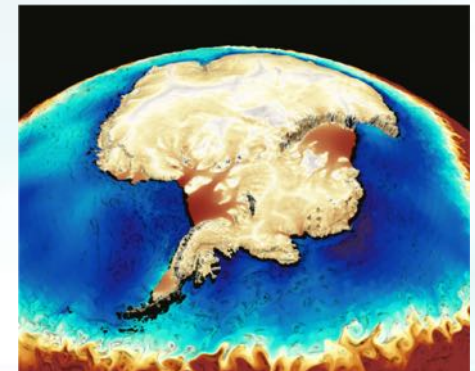
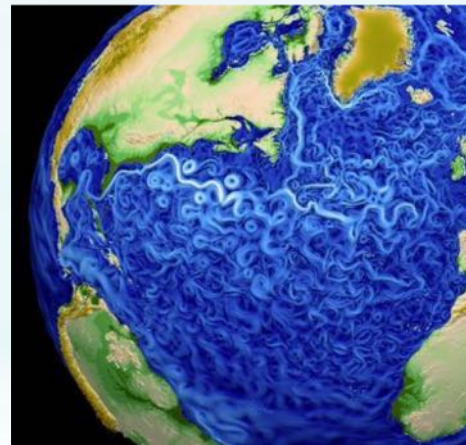
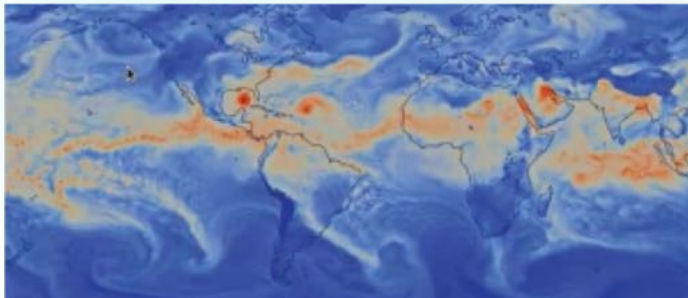
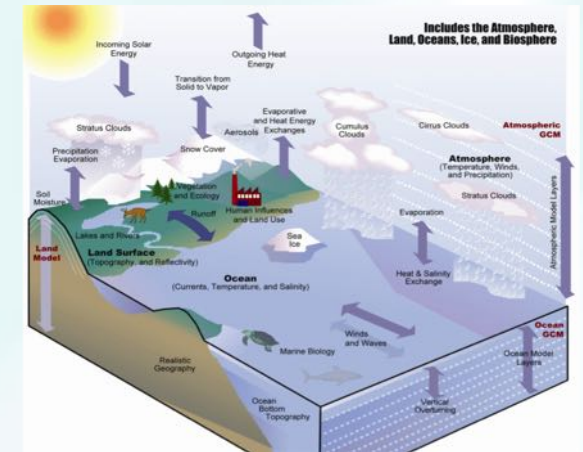
- Jed Brown

Overview

- GPU systems for climate modeling?
 - Traditional climate simulation campaigns require running in the strong scaling limit for throughput
 - In the strong scaling regime, will any new machine be able to outperform the 2012 Edison (IvyBridge)?
 - Other regimes where GPUs can be competitive?
- Fortran vs C++/Kokkos

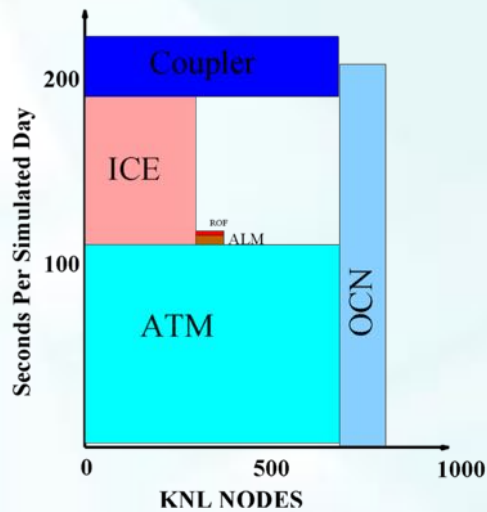
E3SM: Energy Exascale Earth System Model

- Total ~53 FTEs spread over 87 staff involving 8 laboratories
- Atmosphere, Land, Ocean and Ice component models
- Development driven by DOE-SC mission interests: Energy/water issues looking out 40 years
- **Key computational goal: Ensure E3SM will run well on next generation Exascale architectures.**
- Open source: <http://e3sm.org>

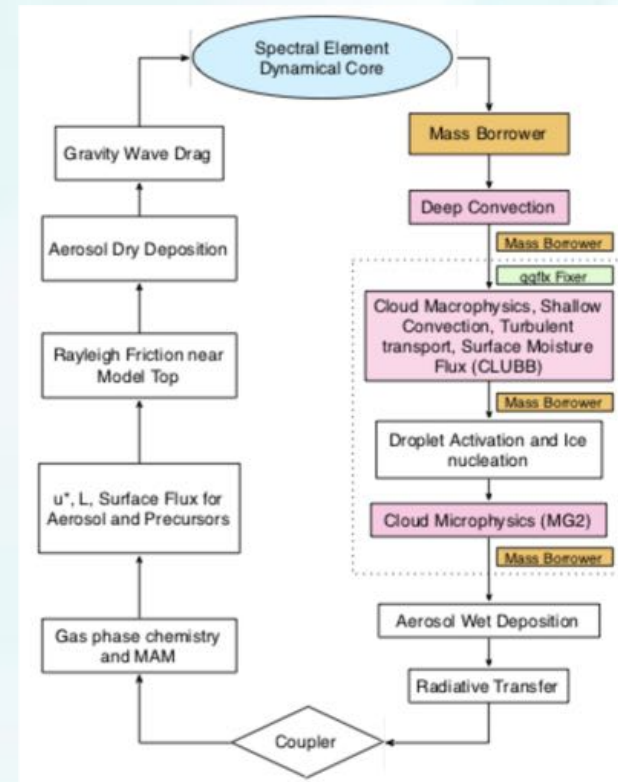


Atmosphere Performance

- Atmosphere is the most expensive component
- Physics (52%) is spread over dozens of parameterizations
- Dycore (transport + dynamics, 48% total) has the most work per lines of code and is usually our first porting/acceleration target

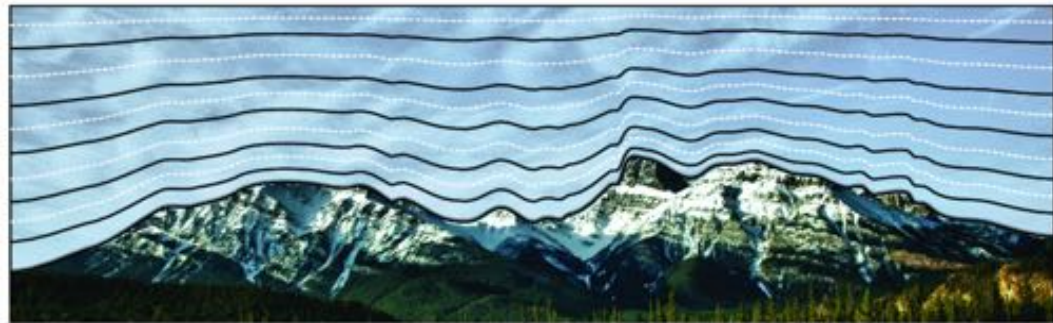
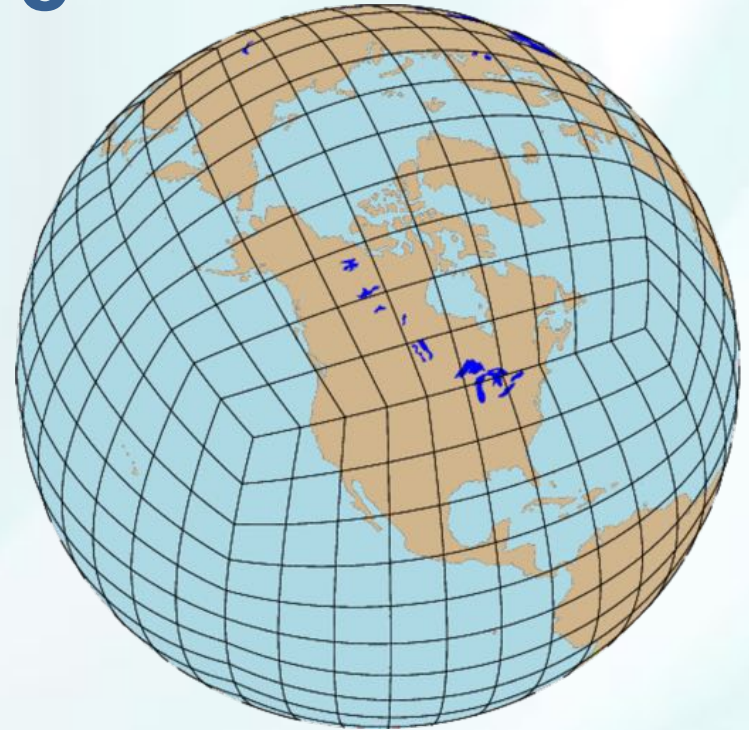


EAM Component	% time
Transport	0.35
Dynamics	0.13
Physics/Chem	0.52



HOMME Dynamical core

- Euler equations w/ shallow atmosphere approximation
- Lorenz staggering in the vertical
- Mimetic vertical differencing (Simmons & Burridge MWR 1981)
- SE mimetic horizontal differencing (Taylor & Fournier, JCP, 2010)
- Vertically Lagrangian (SJ Lin, MWR 2004)

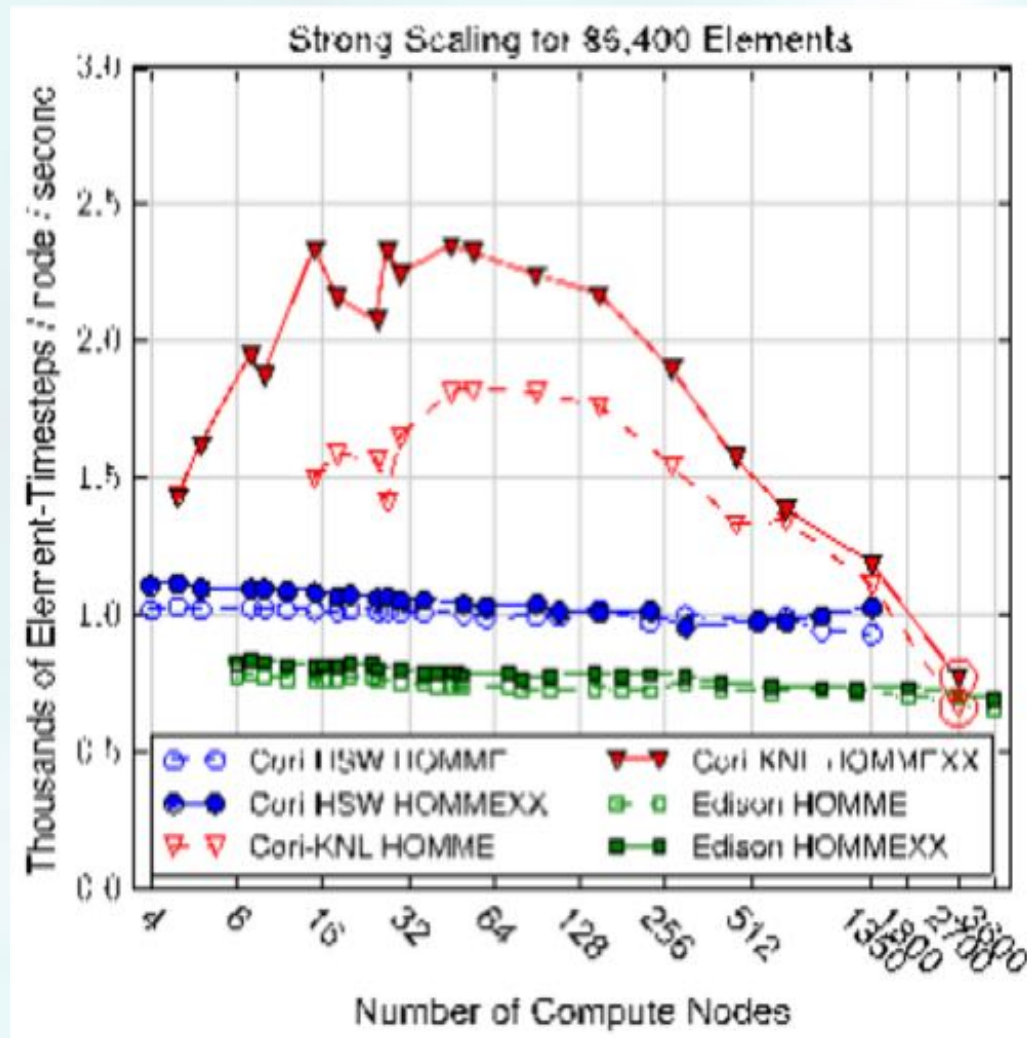


hydrostatic-pressure terrain-following coordinates

Fortran vs C++/kokkos

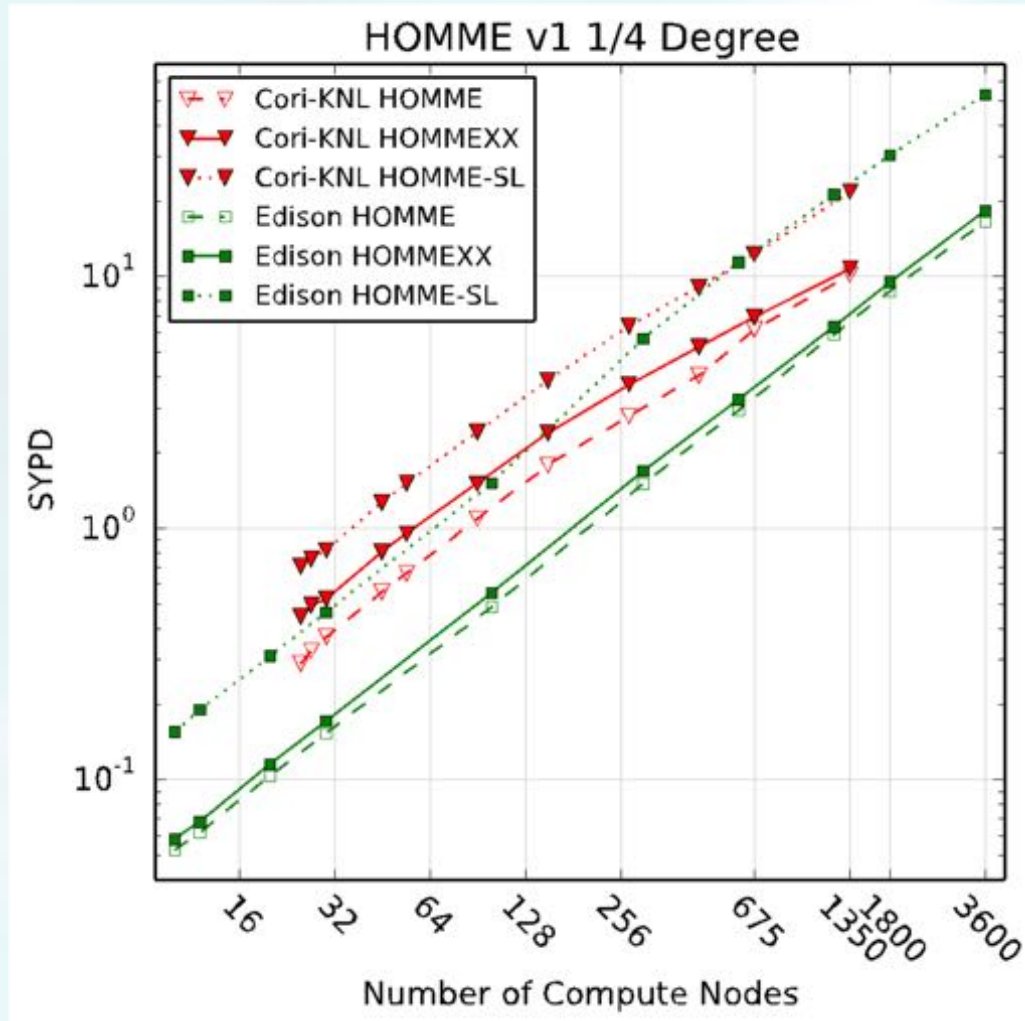
HOMMEXX: C++/Kokkos implementation

- HOMME rewritten from scratch in C++ using the Kokkos performance portable array class
- True performance portability:
- Single code base outperforms the Fortran version on CPU (Xeon) and multicore (KNL) systems, across all workloads
- And: runs well on GPU systems
- C++/Kokkos slightly faster than Fortran on all platforms at all workloads



Atmosphere Dycore + 40 tracers E3SM v1 configuration

- HOMMEXX: slightly faster than HOMME on Edison and KNL
- Biggest performance increases come from SL transport
- With SL transport, both KNL and Edison can achieve throughput (20 SYPD) for the coupled model to run at 5 SYPD



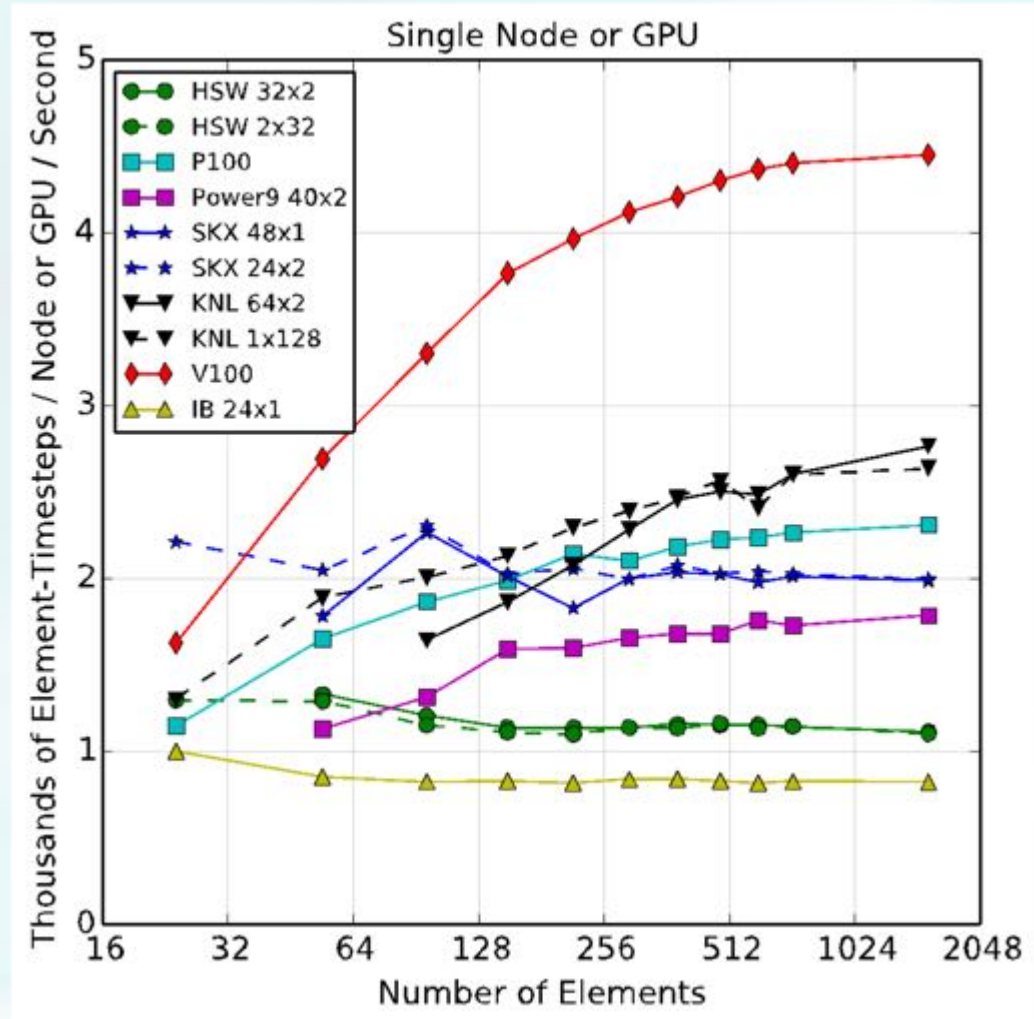
Single Node Performance

Power Consumption

- Compare dual socket nodes vs single GPU
- Roughly equivalent in terms of power consumption:
 - Xeon (Ivy Bridge) node: 260W
 - Xeon (Haswell) node: 360W
 - Xeon (Skylake) node: 330W
 - KNL Node: 260W
 - V100 (per GPU): 260W

Atmosphere Dycore + 40 tracers Single Node/GPU Performance

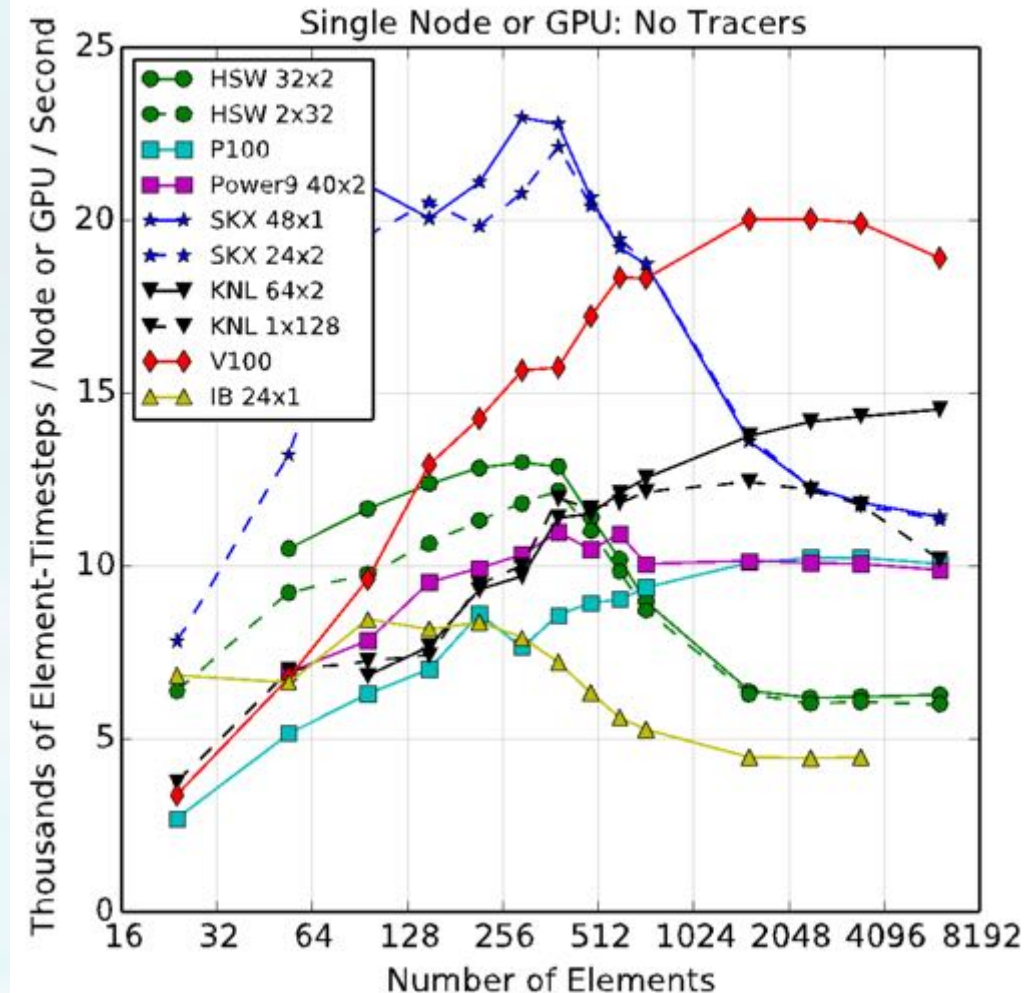
- Best case results. Illustrates several trends:
- Moore's Law - near death
- New architectures (GPU and KNL) only provide speedup in the high work/node regime.
- High throughput (5 SYDP): GPU/KNL not helpful.
- Medium throughput (1 SYDP): moderate benefit
- Low throughput ($\ll 1$ SYDP): can provide significant benefit but GPU requires significant code investment



Atmosphere Dycore – dynamics only

Single node/GPU Performance

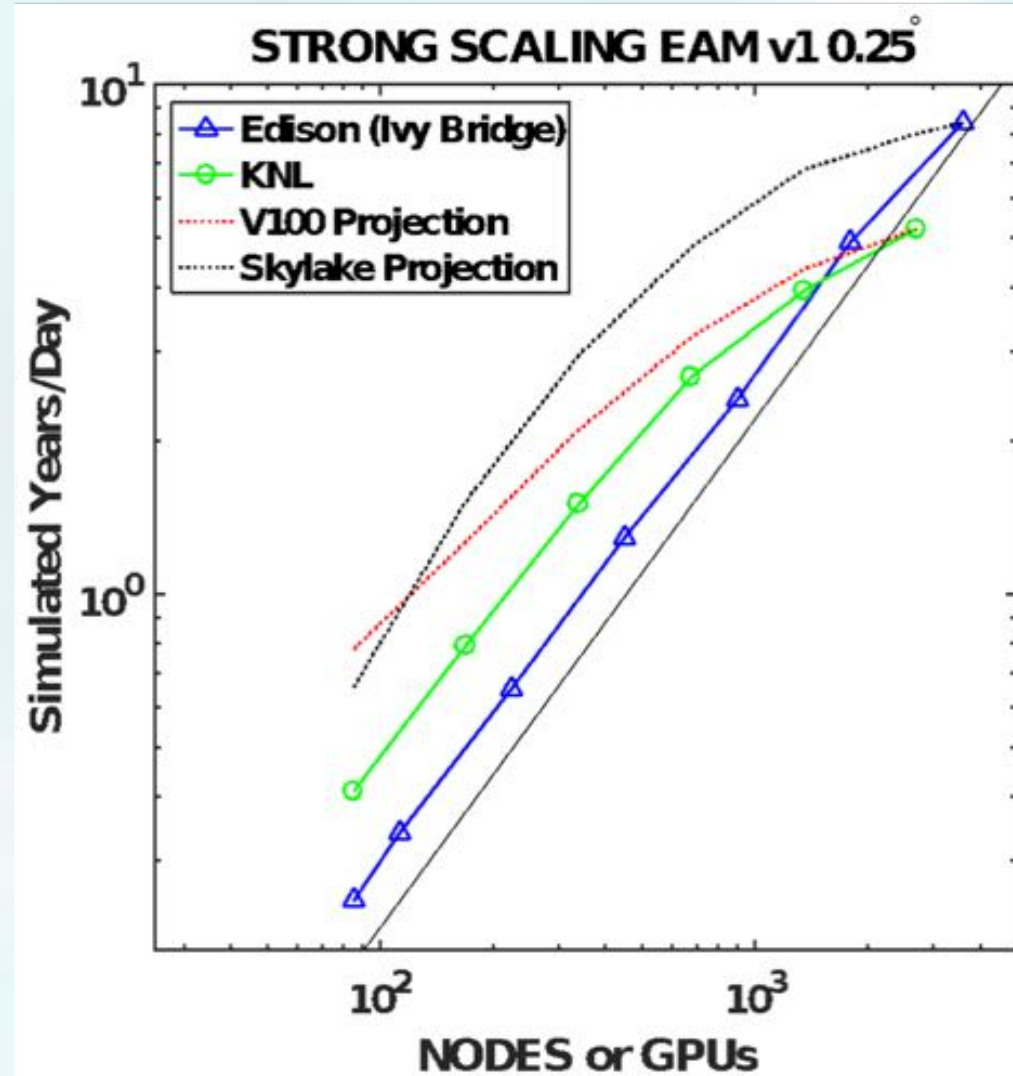
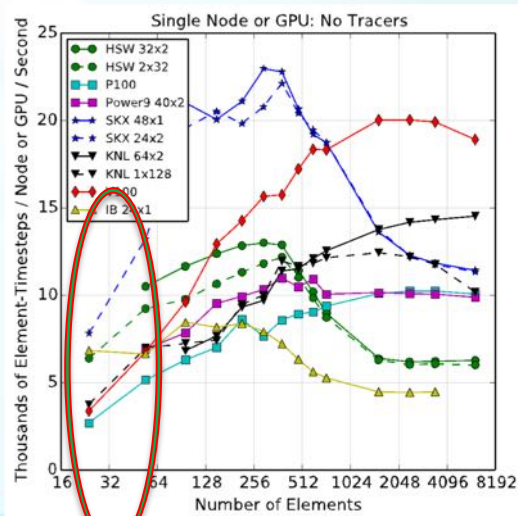
- Dynamics only, no tracers
- More typical of performance we expect from most E3SM subcomponents



Full Atmosphere Model High Resolution Climate (25km)

Atmosphere Performance

- Full EAM model, v1 physics
- 28km,72L, 40 tracers
- Only with sufficient work per node can KNL outperform Edison (Xeon Ivy Bridge)
- Skylake and GPU (V100) Projections based on benchmarked dycore single node performance



Conclusions

- C++/Kokkos performance portable and can be competitive with Fortran
- Detailed benchmarks give best case scenario for atmosphere dycore performance on KNL and upcoming GPU systems
- GPUs (and KNL):
 - For reasonable work per node: not competitive with CPUs
 - Useful for specialized configurations with large workloads which don't impact CFL:
 - $O(100)$ tracers
 - Ultra high “cloud permitting” resolution for short “process studies”
 - Super-parameterization

CEED Collaborations

- Timestepping: Larger timesteps for explicit methods:
 - Some success with high stage methods optimized for CFL (ARKode/SUNDIALS)
 - High stage methods which expose additional parallelism
 - Lagrangian methods (semi-Lagrangian, ALE)
- Solvers
 - Optimal Helmholtz solvers for high order methods
 - Budget is tight: must be faster than 30 explicit timesteps
- Dissipation
 - More efficient replacements for hyperviscosity
 - “partial assembly”?